

location (such as a file). Redirection can be used for input as well as output, redirecting a file to a command for input. This section describes what you need to do to use redirection in your shell scripts.

Output redirection

The most basic type of redirection is sending output from a command to a file. The Bash shell uses the greater-than symbol (`>`) for this:

```
command> outputfile
```

Anything that would appear on the monitor from the command instead is stored in the output file specified:

```
$ date> test6
$ ls -l test6
-rw-r--r--  1 user      user      29 Jun 01 16:56 test6
$ cat test6
Mon Jun 01 16:56:58 EDT 2020
$
```

The redirect operator created the file `test6` (using the default `umask` settings) and redirected the output from the `date` command to the `test6` file. If the output file already exists, the redirect operator overwrites the existing file with the new file data:

```
$ who> test6
$ cat test6
rich      pts/0      Jun 01 16:55
$
```

Now the contents of the `test6` file contain the output from the `who` command.

Sometimes, instead of overwriting the file's contents, you may need to append output from a command to an existing file — for example, if you're creating a log file to document an action on the system. In this situation, you can use the double greater-than symbol (`>>`) to append data:

```
$ date>> test6
$ cat test6
rich      pts/0      Jun 01 16:55
Mon Jun 01 17:02:14 EDT 2020
$
```

The `test6` file still contains the original data from the `who` command processed earlier — plus now it contains the new output from the `date` command.

Input redirection

Input redirection is the opposite of output redirection. Instead of taking the output of a command and redirecting it to a file, input redirection takes the content of a file and redirects it to a command.

The input redirection symbol is the less-than symbol (`<`):

```
command < inputfile
```

The easy way to remember this is that the command is always listed first in the command line, and the redirection symbol “points” to the way the data is flowing. The less-than symbol indicates that the data is flowing from the input file to the command.

Here's an example of using input redirection with the `wc` command:

```
$ wc < test6
  2      11      60
$
```

The `wc` command provides a count of text in the data. By default, it produces three values:

The number of lines in the text

The number of words in the text

The number of bytes in the text

By redirecting a text file to the `wc` command, you can get a quick count of the lines, words, and bytes in the file. The example shows that there are 2 lines, 11 words, and 60 bytes in the `test6` file.

There's another method of input redirection, called *inline input redirection*. This method allows you to specify the data for input redirection on the command line instead of in a file. This may seem somewhat odd at first, but there are a few applications for this process (such as those shown in the “Performing Math” section later).

The inline input redirection symbol is the double less-than symbol (`<<`). Besides this symbol, you must specify a text marker that delineates the beginning and end of the data used for input. You can use any string value for the text marker, but it must be the same at the beginning of the data and the end of the data:

```
command << marker
data
marker
```

When using inline input redirection on the command line, the shell will prompt for data using the secondary prompt, defined in the `PS2` environment variable (see [Chapter 6](#)). Here's how this looks when you use it:

```
$ wc << EOF
> test string 1
> test string 2
> test string 3
> EOF
  3      9      42
$
```

The secondary prompt continues to prompt for more data until you enter the string value for the text marker. The `wc` command performs the line, word, and byte counts of the data supplied by the inline input redirection.

Employing Pipes

There are times when you need to send the output of one command to the input of another command. This is possible using redirection, but somewhat clunky:

```
$ rpm -qa> rpm.list
$ sort < rpm.list
abattis-cantarell-fonts-0.0.25-1.el7.noarch
abrt-2.1.11-52.el7.centos.x86_64
abrt-addon-ccpp-2.1.11-52.el7.centos.x86_64
abrt-addon-kerneloops-2.1.11-52.el7.centos.x86_64
abrt-addon-pstoreoops-2.1.11-52.el7.centos.x86_64
abrt-addon-python-2.1.11-52.el7.centos.x86_64
abrt-addon-vmcore-2.1.11-52.el7.centos.x86_64
abrt-addon-xorg-2.1.11-52.el7.centos.x86_64
abrt-cli-2.1.11-52.el7.centos.x86_64
abrt-console-notification-2.1.11-52.el7.centos.x86_64
...
```

The `rpm` command manages the software packages installed on systems using the Red Hat Package Management system (RPM), such as the CentOS system as shown. When used with the `-qa` parameters, it produces a list of the existing packages installed, but not necessarily in any specific order. If you're looking for a specific package, or group of packages, it can be difficult to find it using the output of the `rpm` command.

Using the standard output redirection, the output was redirected from the `rpm` command to a file called `rpm.list` . After the command finished, the `rpm.list` file contained a list of all the installed software packages on this system. Next, input redirection was used to send the contents of the `rpm.list` file to the `sort` command to sort the package names alphabetically.

That was useful, but again, a somewhat clunky way of producing the information. Instead of redirecting the output of a command to a file, you can redirect the output to another command. This process is called *pip*ing.

Like the command substitution backtick (```), the symbol for piping is not used often outside of shell scripting. The symbol is two vertical lines, one above the other. However, the pipe symbol often looks like a single vertical line in print (`|`). On a U.S. keyboard, it is usually on the same key as the backslash (`\`). The pipe is put between the commands to redirect the output from one to the other:

```
command1 | command2
```

Don't think of piping as running two commands back to back. The Linux system actually runs both commands at the same time, linking them together internally in the system. As the first command produces output, it's sent immediately to the second command. No intermediate files or buffer areas are used to transfer the data.

Now, using piping you can easily pipe the output of the `rpm` command directly to the `sort` command to produce your results:

```
$ rpm -qa | sort
abattis-cantarell-fonts-0.0.25-1.el7.noarch
abrt-2.1.11-52.el7.centos.x86_64
abrt-addon-ccpp-2.1.11-52.el7.centos.x86_64
abrt-addon-kerneloops-2.1.11-52.el7.centos.x86_64
abrt-addon-pstoreoops-2.1.11-52.el7.centos.x86_64
abrt-addon-python-2.1.11-52.el7.centos.x86_64
abrt-addon-vmcore-2.1.11-52.el7.centos.x86_64
abrt-addon-xorg-2.1.11-52.el7.centos.x86_64
abrt-cli-2.1.11-52.el7.centos.x86_64
abrt-console-notification-2.1.11-52.el7.centos.x86_64
...
```

Unless you're a (very) quick reader, you probably couldn't keep up with the output generated by this command. Because the piping feature operates in real time, as soon as the `rpm` command produces data, the `sort` command gets busy sorting it. By the time the `rpm` command finishes outputting data, the `sort` command already has the data sorted and starts displaying it on the monitor.

There's no limit to the number of pipes you can use in a command. You can continue piping the output of commands to other commands to refine your operation.

In this case, because the output of the `sort` command zooms by so quickly, you can use one of the text paging commands (such as `less` or `more`) to force the output to stop at every screen of data:

```
$ rpm -qa | sort | more
```

This command sequence runs the `rpm` command, pipes the output to the `sort` command, and then pipes that output to the `more` command to display the data, stopping after every screen of information. This now lets you pause and read what's on the display before continuing, as shown in [Figure 11-1](#).

To get even fancier, you can use redirection along with piping to save your output to a file:

```
$ rpm -qa | sort> rpm.list
$ more rpm.list
abrt-1.1.14-1.fc14.i686
abrt-addon-ccpp-1.1.14-1.fc14.i686
abrt-addon-kerneloops-1.1.14-1.fc14.i686
abrt-addon-python-1.1.14-1.fc14.i686
abrt-desktop-1.1.14-1.fc14.i686
abrt-gui-1.1.14-1.fc14.i686
abrt-libs-1.1.14-1.fc14.i686
abrt-plugin-bugzilla-1.1.14-1.fc14.i686
abrt-plugin-logger-1.1.14-1.fc14.i686
abrt-plugin-runapp-1.1.14-1.fc14.i686
acl-2.2.49-8.fc14.i686
...
```

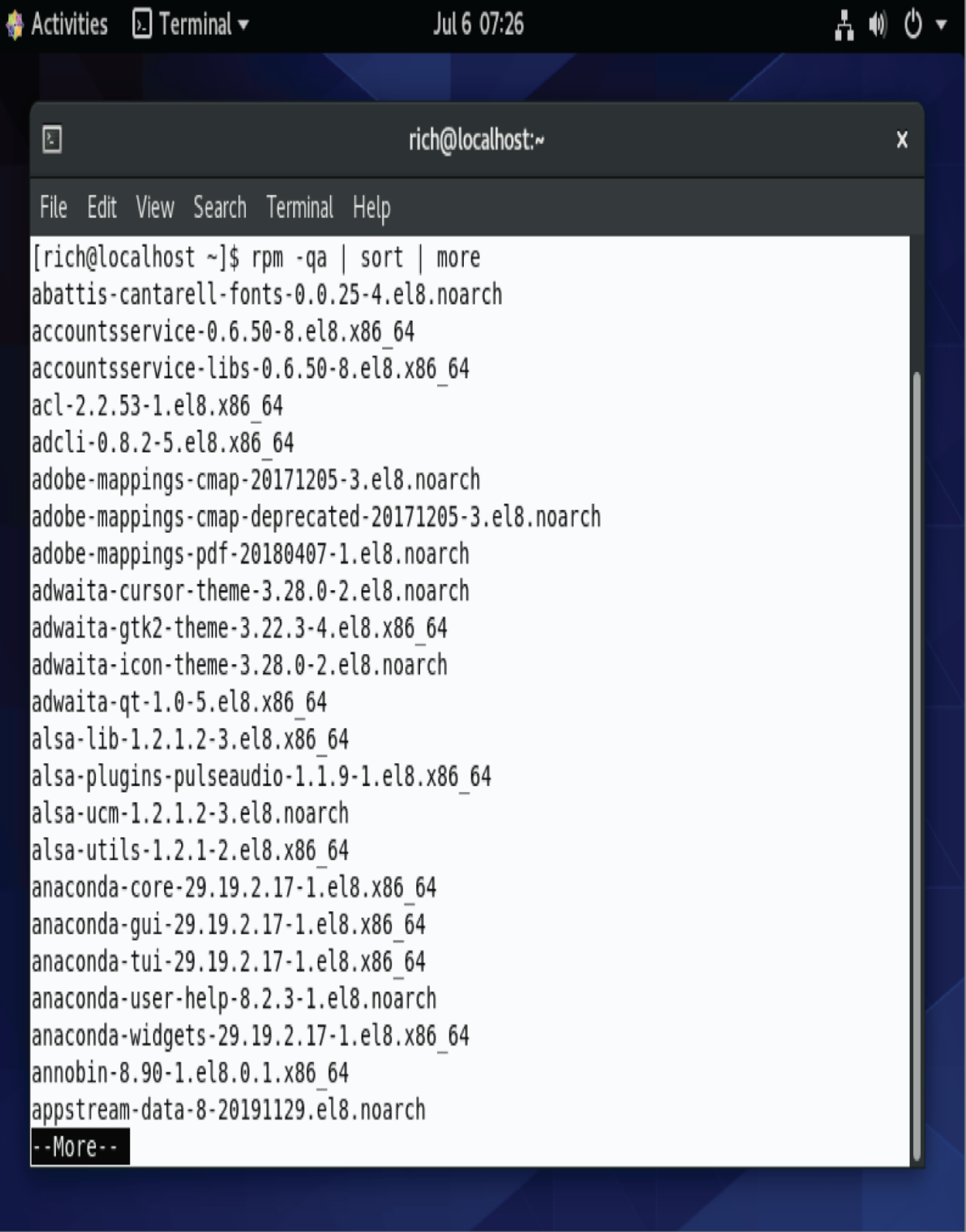


FIGURE 11-1 Using piping to send data to the `more` command

As expected, the data in the `rpm.list` file is now sorted!

By far one of the most popular uses is piping the results of commands that produce long output to the `more` command. This is especially common with the `ls` command, as shown in [Figure 11-2](#).

The `ls -l` command produces a long listing of all the files in the directory. For directories with lots of files, this can be quite a listing. By piping the output to the `more` command, you force the output to stop at the end of every screen of data.

Performing Math

Another feature crucial to any programming language is the ability to manipulate numbers. Unfortunately, for shell scripts this process is a bit awkward. You have two ways to perform mathematical operations in your shell scripts.